



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

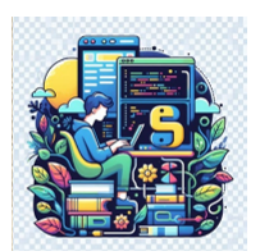
PROGRAMMING WITH PYTHON

Object-Oriented Programming (OOP) and Advanced Concepts

Static and Class Methods

Lekha A

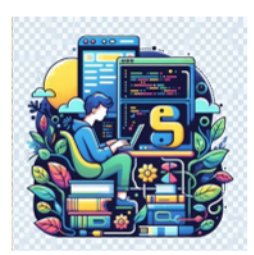
Computer Applications



PROGRAMMING WITH PYTHON

Static and Class Methods

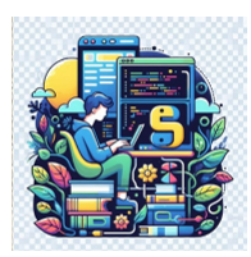
- Static and class methods are two special types of methods in Python that are associated with classes rather than individual objects.
- They are both defined using decorators, which are special functions that add functionality to other functions or classes.



PROGRAMMING WITH PYTHON

Static Methods

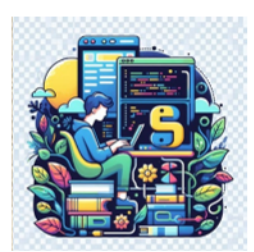
- A static method is a method that doesn't operate on the instance or class itself.
- It's independent of the class and does not have access to the class or instance attributes.
- Define a static method using the `@staticmethod` decorator.



PROGRAMMING WITH PYTHON

Static Methods

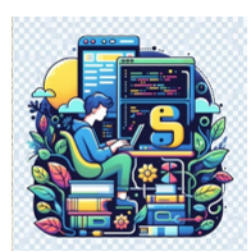
- Static methods are used for utility functions that are related to the class but do not operate on specific instances of the class.
- These utility functions are often related to the class as a whole, such as providing constants or performing calculations.



PROGRAMMING WITH PYTHON

Static Methods

- Static methods in Python are often used when a method in a class doesn't need access to the instance or the class itself.
- They are independent of the instance variables or class variables and can be called without creating an instance of the class.
- One real-time application of a static method could be in a utility function that doesn't require any information from the class or its instances.



PROGRAMMING WITH PYTHON

Static Methods

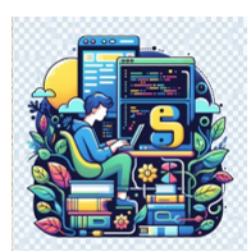
```
class Math:  
    @staticmethod  
    def pi():  
        return 3.14159  
  
    @staticmethod  
    def is_even(number):  
        return number % 2 == 0
```

```
res=Math.is_even(25)  
res
```

False

```
res=Math.pi()*5  
res
```

15.70795



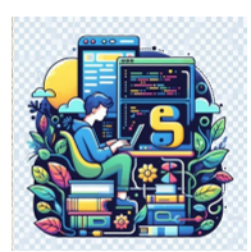
PROGRAMMING WITH PYTHON

Static Methods

```
class MyClass:  
    class_variable = 10  
  
    def __init__(self, value):  
        self.instance_variable = value  
  
    @staticmethod  
    def static_method(x, y):  
        return x + y
```

```
result = MyClass.static_method(5, 3)
```

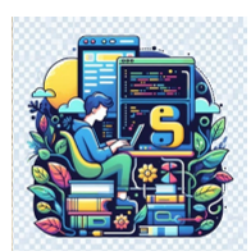
```
result
```

PROGRAMMING WITH PYTHON

Class Methods

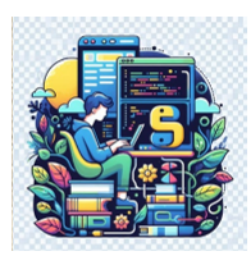
- Class methods are defined using the `@classmethod` decorator.
- They are bound to the class itself, not to individual objects.
- They have access to the class's state (e.g., `cls`) but not to individual objects (e.g., `self`).
- Class methods are typically used for tasks that involve modifying the class's state or creating new instances of the class.



PROGRAMMING WITH PYTHON

Class Methods

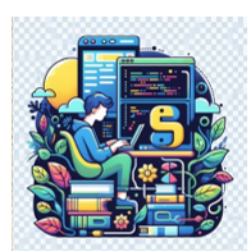
- Class methods in Python are used when a method needs access to the class itself, rather than an instance of the class.
- They take the class itself (often referred to as `cls`) as the first argument, allowing the modification of class variables or access other class-specific information.



PROGRAMMING WITH PYTHON

Class Methods

- Class methods are used for tasks that involve modifying the class's state or creating new instances of the class.
- These tasks typically require access to the class itself, not to individual objects of the class.



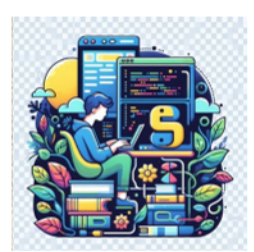
PROGRAMMING WITH PYTHON

Class Methods

```
class MyClass:
    class_variable = 10

    def __init__(self, value):
        self.instance_variable = value

    @classmethod
    def class_method(cls, x):
        cls.class_variable += x
```



PROGRAMMING WITH PYTHON

Class Methods

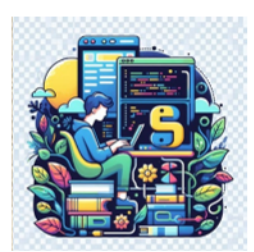
- `obj1 = MyClass(5)`

`obj1.class_method(2)``print(obj1.class_variable)`

12

`obj2 = MyClass(8)``obj2.class_method(3)``print(obj2.class_variable)`

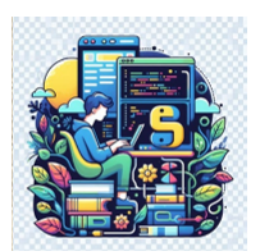
15



PROGRAMMING WITH PYTHON

Static vs. Class Methods

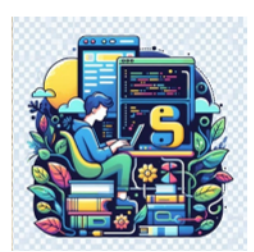
Static Methods	Class Methods
Not bound to objects or the class state	Bound to the class, not objects
Used for utility functions related to the class	Have access to the class state (cls)
Don't have access to self or cls	Can be used to modify the class's state or create new instances
	Can also be called using the class name (ClassName.method())



PROGRAMMING WITH PYTHON

`__new__()`

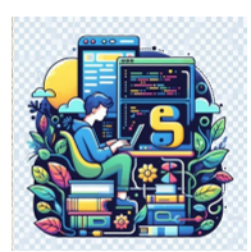
- The `__new__()` method is a special static method used to create a new instance of a class.
- Is part of Python's object creation process and is called before `__init__()`.
- It is responsible for allocating memory and returning a new instance of the class.



PROGRAMMING WITH PYTHON

`__new__()`

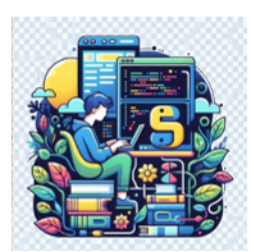
- After the instance is created, Python calls the `__init__()` method to initialize it.
- `cls` is used as the parameter name to signify that it refers to the class that is being instantiated.
- The purpose of `__new__` is to allocate memory for a new object.
 - To allocate memory, it needs to know which class the object belongs to.



PROGRAMMING WITH PYTHON

`__new__()`

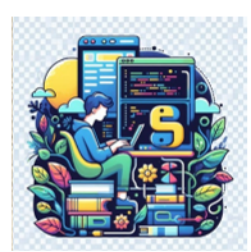
- At the time `__new__` is called, the instance does not yet exist.
- Therefore, `__new__` requires the class itself (`cls`) to create and return a new instance.



PROGRAMMING WITH PYTHON

__new__ vs __init__

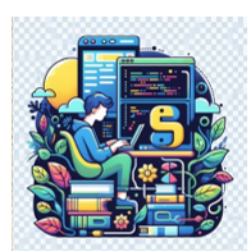
	__new__	__init__
Method	Static	Instance
Functionality	responsible for creating and returning a new instance	responsible for initializing the attributes of the newly created object.
Call	Before __init__	
Return	return any object	return None.



PROGRAMMING WITH PYTHON

When to use?

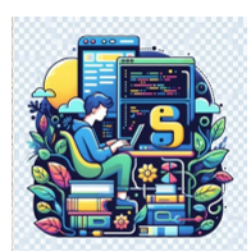
- `__new__`
 - To control the creation of the object.
 - Scenarios
 - Ensure that the object is of a certain type.
 - Set the object's initial state.
 - Prevent the object from being created.



PROGRAMMING WITH PYTHON

When to use?

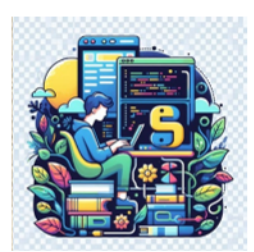
- `__init__`
 - To initialize the object.
 - Scenarios
 - Set the object's attributes.
 - Call the object's superclass' `__init__` method.
 - Perform other initialization tasks.



PROGRAMMING WITH PYTHON

When to use static and class methods

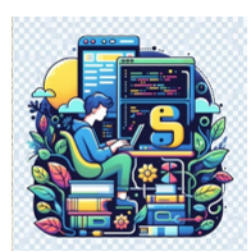
- Use static methods when there is a need for a utility function that is related to the class but does not operate on specific instances of the class.
- Use class methods when there is a need to perform a task that involves modifying the class's state or creating new instances of the class.



PROGRAMMING WITH PYTHON

Benefits of static and class methods

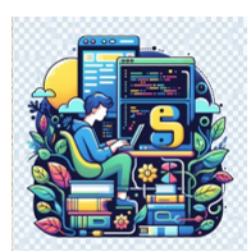
- Improved code organization
 - Static and class methods help to organize code by keeping utility functions and class-related tasks separate from instance methods.
- Encapsulation
 - They enforce encapsulation by restricting access to the class's state and preventing modifications to it from outside the class.



PROGRAMMING WITH PYTHON

Benefits of static and class methods

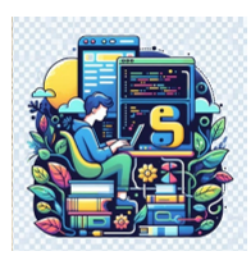
- Flexibility
 - They allow developer to add functionality to classes without modifying their existing interface.



PROGRAMMING WITH PYTHON

Advantages of static methods

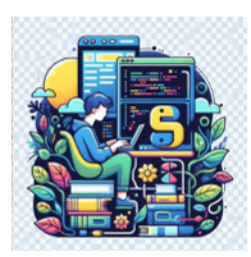
- Utility Functions
 - They are used for functions that do not require access to instance or class data.
 - These functions can be relevant to the class but don't operate on instance-specific or class-specific attributes.



PROGRAMMING WITH PYTHON

Advantages of static methods

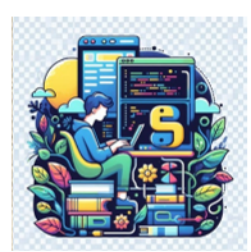
- Code Organization
 - Static methods help in organizing code within a class, grouping related functionality under the class name, even if the method does not rely on instance or class variables.



PROGRAMMING WITH PYTHON

Advantages of static methods

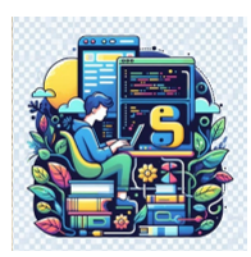
- Readability
 - They improve code readability by making it clear that the method is relevant to the class but does not rely on instance-specific data.



PROGRAMMING WITH PYTHON

Advantages of static methods

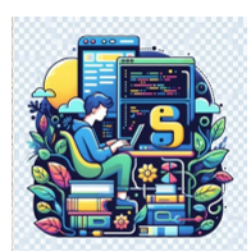
- Accessing Class-Specific Data
 - Class methods have access to the class itself via the cls parameter.
 - They can modify or access class-specific attributes, which makes them useful for modifying class-level attributes or creating factory methods.



PROGRAMMING WITH PYTHON

Advantages of class methods

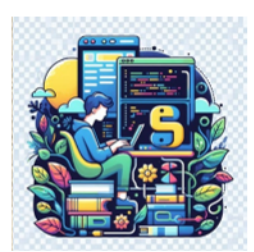
- Alternate Constructors (Factory Methods)
 - Class methods are often used as alternate constructors to create instances of the class using different parameters or initializations.



PROGRAMMING WITH PYTHON

Advantages of class methods

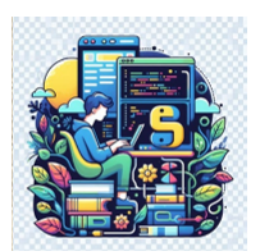
- Inheritance and Polymorphism
 - Class methods are inherited by subclasses, allowing them to be overridden and enabling polymorphism.
 - This facilitates uniform behavior across different classes.



PROGRAMMING WITH PYTHON

Use Cases

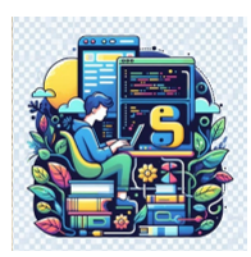
- Static Methods
 - For common utility functions that are relevant to a class but don't need access to instance or class data.
- Example
 - Mathematical operations, string formatting or data transformations that are relevant to the class but don't depend on specific instances.



PROGRAMMING WITH PYTHON

Use Cases

- Class Methods
 - For factory methods, alternate constructors, or methods that modify class attributes.
 - For instance, initializing class variables, manipulating class-specific data, or providing controlled instance creation.



PROGRAMMING WITH PYTHON

Recap

- Static Methods
- Class Methods
- Static vs. Class Methods
- When to Use Static and Class Methods
- Benefits of Static and Class Methods
- Advantages of Static Methods
- Advantages of Class Methods
- Use Cases



THANK YOU

Lekha A
Computer Applications